

Exam-2 Program Verification (OLD, for exercise only)

Lecturers: Wishnu Prasetya

SPIN/Promela

1. Consider a system of N traders implemented as programs. N is some number ≥ 3 . A trader cycles between being idle and doing its work, which is trading. To do trading, a trader needs two resources: one of type B (bank) and one of type S (seller). It does not matter which instances of B and S it gets, as long as it gets a pair of them.

To use a resource, a trader must lock it first. There are 3 resources of type B and 3 resources of type S in the system.

A Promela model of such this system is shown below. The variable $lockB$ and $lockS$ are used to model which trader currently locks a resource. If $i \geq 0$, $lockB[k] == i$ means that a trader, namely trader i , currently locks resource B_k . Else, $lockB[k] < 0$ means that B_k is free. The interpretation of $lockS$ is similar.

Trader i will set $working[i]$ to true when it is busy with trading. When it is done, $working[i]$ is set to false again.

```
#define FREE -1
#define IDLE 0
#define WORKING 1
#define N = ... // some number >= 3
byte lockB[3] = FREE ; // the locks to the B-resources, all initially are free
byte lockS[3] = FREE ; // the locks to the S-resources, all initially are free
bool working[N] = IDLE // working-i is true when trader-i is trading, else false

proctype trader(byte i) {
    byte b ; byte s ;
    do // cycle forever, doing work
    :: {
        // acquire the needed resources:
        ...
        ... // to be completed in (d)
        ...
        assert lockB[b]==i && lockS[s]==i ;
        // start trading:
        working[i] = WORKING ;
        // trading is finished
        working[i] = IDLE ;
        lockB[b] = FREE ; lockS[s] = FREE ;
    }
    od
}

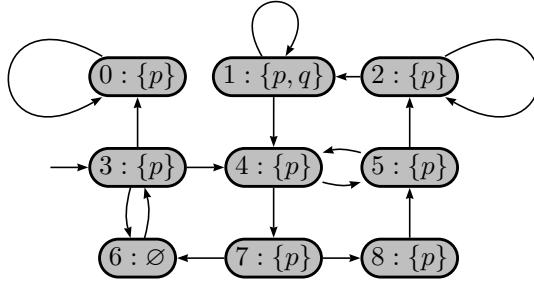
init {
    byte i = 0 ;
    atomic {
        do
            :: i<N -> run trader(i) ; i = i+1 ;
            :: i==N -> break ;
        od }
}
```

Your tasks:

- (a) Write an LTL formula that means that our system of traders is deadlock-free.
- (b) In analogy to the Dining Philosophers problem, imagine that a trader would 'starve' if it never gets a turn to do trading. Write an LTL formula stating that our system of traders is starvation-free.
- (c) Suppose we do SPIN's 'invalid end-state' check on the model above, and it report **no** violation. Does this imply that the model is thus starvation free? Explain.
- (d) The model above is incomplete as it leaves out how a trader can acquire the resources it needs. Propose your solution for this, taking the following into account:
 - i. It should be deadlockfree.
 - ii. In the actual system, it is possible to inspect whether a resource is free, and to subsequently lock it in a single 'atomic' execution (that is, without other traders change the state of the resource in-between).
 - iii. However, locking multiple resources *cannot* be done in a single atomic execution.
 - iv. We want a solution that does not choke the parallelism between the traders.

CTL Model Checking

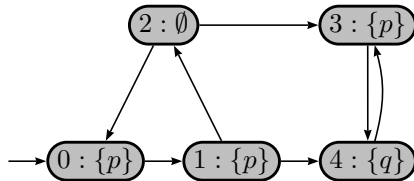
2. Consider the following program, represented by a Kripke structure M ; 3 is the initial state, and $Prop = \{p, q\}$.



- (a) Which states can be labelled with $\mathbf{E}(p \mathbf{U} q)$?
- (b) We want to check if $\phi = \mathbf{AX}(\mathbf{E}(p \mathbf{U} q))$ is valid on M . Describe the steps to do this checking. So, is ϕ valid?

Symbolic Model Checking

3. Consider a Kripke structure M shown below. It has four states: $s_0, \dots, s_4$, with s_0 as the initial state. We have $Prop = \{p, q\}$. Each state is labelled by the set of properties that hold on the state. When a property is not listed as a label of a state s , it means that the property does not hold on s .



Suppose we use the following symbolic encoding of M 's states. We write fg to abbreviate $f \wedge g$.

state	encoding
s_0	$\neg a \neg b \neg c$
s_1	$\neg a \neg b c$
s_2	$\neg a b \neg c$
s_3	$\neg a b c$
s_4	$a \neg b \neg c$

Your tasks:

- (a) Let W_p be the set of states of M where p holds. Give the symbolic representation of W_p as a Boolean formula.
 - (b) We want to encode M as a Boolean function $R(a, b, c, a', b', c')$, where the primed variables are used to encode next-states. Give this function R .
 - (c) Show how to construct the symbolic representation of $W_{\mathbf{EX}q}$.
4. Consider the Boolean formula $ab \vee xy$. Again, we write fg to abbreviate $f \wedge g$.
- (a) Give the reduced and ordered BDD of the above formula, using the following variable-ordering: a, b, x, y .
 - (b) Give the reduced and ordered BDD of the above formula. This time, use the following variable-ordering: a, x, b, y .
 - (c) Let G_f and G_g be reduced OBDDs representing a Boolean formula f and g . Question: how to use the OBDDs to check whether $f = g$ is valid (in other words, to check whether f and g are equivalent)?

PCTL

5. Below is the syntax of Probabilistic CTL (PCTL). In the description, a is a proposition, ϕ, ϕ_1, ϕ_2 are PCTL formulas, ψ is a path formula, k is a natural number, and p is a numeric value.

$$\phi ::= a \mid \neg\phi \mid \phi_1 \wedge \phi_2 \mid \mathbf{P}_{\sim p}[\psi] \quad , \text{ where } \sim \text{ is one of } <, \leq, >, \geq, =$$

$$\psi ::= \mathbf{X}\phi \mid \phi_1 \mathbf{U}^{\leq k} \phi_2 \mid \phi_1 \mathbf{U} \phi_2$$

We represent a model (of a program) as a Discrete-time Markov Chain (DTMC) $M = (S, s_0, P, L, Prop)$, where S is a finite set of states, s_0 is the program's initial state, P is a function that describes the probabilistic transitions between the states:

$$P(s, t) = v$$

If $v > 0$ this means that there exists a transition that goes from the state s to the state t , and the probability that it is taken (when the system is in s) is v . If $v = 0$, it means that there is no transition from s to t .

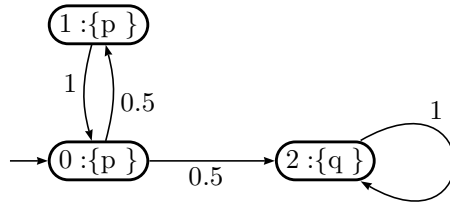
L is the usual labelling function. It labels every state s with a set of propositions from $Prop$ that holds on s , such that when $p \in L(s)$ then p holds on s , and otherwise it does not hold on s .

A PCTL property holds on M , denoted by $M \models \phi$, if it holds on M 's initial state, denoted by $s_0 \models \phi$.

Your task: give a formal definition of $s \models \phi$. For the probabilistic operator $\mathbf{P}_{\sim p}$ you only need to give a definition for $\mathbf{P}_{=p}$ (so, $\sim$ is just "=").

Hint: the definition is recursive over the syntax of ϕ .

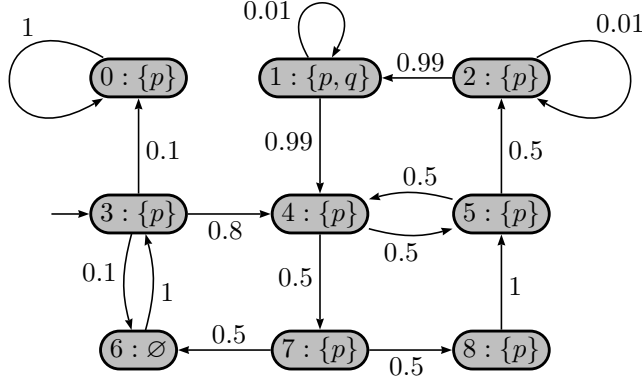
6. Consider the following DTMC M . It has three states: 0,1,2, with 0 as the initial state.



- Consider the execution path $[0, 1, 0, 1, \dots]$ that keeps cycling over the state 0 and 1 indefinitely. What is the probability that this behavior occurs?
- Explain why in CTL $\mathbf{A}(p \mathbf{U} q)$ is **not** a valid property of M , but in PCTL $\mathbf{P}_{\geq 1}[p \mathbf{U} q]$ is valid.

PCTL Model Checking on DTMC

7. Consider the following DTMC M ; 3 is the initial state. It is the same model as in question No. 2, but we now have the probabilities of the transitions.

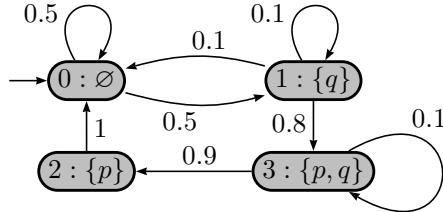


We want to check if $\mathbf{P}_{>0.49}[p \mathbf{U} q]$ holds on the model M . So, in other words to check: $3 \models \mathbf{P}_{>0.49}[p \mathbf{U} q]$

Tasks:

- The set S^{no} is defined as the set $\{s \mid s \in S \text{ such that } s \models \mathbf{P}_{\leq 0}[p \mathbf{U} q]\}$, where S is the set of all states in M . How do you calculate S^{no} ? Which states are in S^{no} ?
- Give the definition of S^{yes} . Which states are in S^{yes} ?
- Calculate the probability of $p \mathbf{U} q$ for every state t which is *not* in S^{yes} nor S^{no} . In other words, calculate: $t \models \mathbf{P}_{=?}[p \mathbf{U} q]$, for every $t \in S/(S^{yes} \cup S^{no})$.
- So, which of these properties are valid?
 - $3 \models \mathbf{P}_{>0.49}[p \mathbf{U} q]$
 - $6 \models \mathbf{P}_{>0.49}[p \mathbf{U} q]$
 - $1 \models \mathbf{P}_{>0.49}[p \mathbf{U} q]$

8. Consider the following DTMC M ; 0 is the initial state:



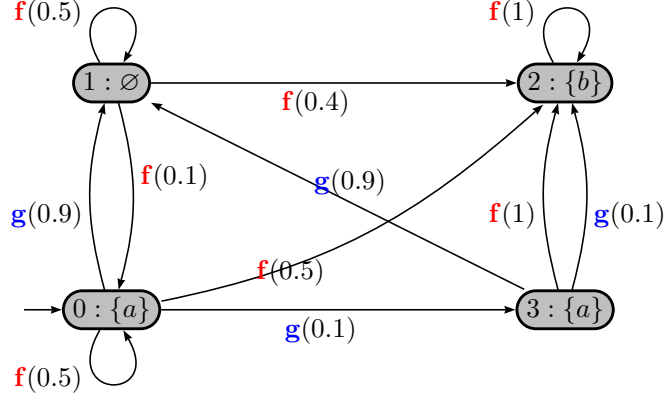
And here is a matrix representation of M . Let $P_{i,j}$ refers to the element in the i -th row and j -th column. It represents the probability of the transition from state i to state j . E.g. $P_{3,2} = 0.9$ is the probability of the transition $3 \rightarrow 2$ in M .

$$P = \begin{bmatrix} 0.5 & 0.5 & 0 & 0 \\ 0.1 & 0.1 & 0 & 0.8 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0.9 & 0.1 \end{bmatrix}$$

Use matrix calculation to check whether the probability to reach q from the initial state in exactly **two** steps is at least 0.7. (show the calculation)

PCTL Model Checking on MDP

9. Consider the following DTMC M ; 3 is the initial state. $Prop = \{a, b\}$.



The model can do two possible actions: f or g . Which actions are possible at every state are indicated in the picture above. Every arrow also shows the probability that it is taken, when the corresponding action is selected to be executed. Notice that at state 0 and state 3, M has two actions to choose: f or g .

We want to verify if:

$$\mathbf{P}_{>0.1}[a \mathbf{U} b]$$

holds on M .

In other words, we want to check whether $p_{\min}(0, a \mathbf{U} b) > 0.1$ holds (note that "0" here refers to state 0, the initial state).

- Explain, what is $p_{\min}$?
- Give the definition of S^{yes} and S^{no} .
- Which states are in S^{yes} and respectively S^{no} ?
- Does $p_{\min}(0, a \mathbf{U} b) > 0.1$ hold? Show how you check this.